CS 651: WEB SYSTEMS · CALIFORNIA STATE UNIVERSITY, EAST BAY

VocalIQ

AI Speaking Coach

Analytics & Logging Report

Team Members Angaddeep Singh Gupta · Abhinay Konuri

Samuel Paul Chetty · Lasya Uma Sri Lingala

Project 2 · Spring 2026

Section 1: Google Analytics

1.0 Implementation Summary

VocalIQ implements Google Analytics 4 (GA4) at two distinct levels — client-side via the **gtag.js** library and server-side via the **GA4 Measurement Protocol** — providing a complete view of user behavior from both the browser and the backend.

GA4 Measurement ID: G-SN3LE9Y5RT

Client-Side (React.js): The gtag.js snippet is loaded in `client/index.html`, which initializes GA4 on every page load. A custom `analytics.js` utility module exposes two functions — `trackPageView(path)` and `trackEvent(eventName, params)` — which are called throughout the React application. A `PageViewTracker` component placed inside React Router fires a page view on every route change automatically.

```
// client/src/utils/analytics.js
const GA_ID = "G-SN3LE9Y5RT";

export const trackPageView = (path) => {
  if (typeof window.gtag !== "function") return;
  window.gtag("config", GA_ID, { page_path: path });
};

export const trackEvent = (eventName, eventParams = {}) => {
  if (typeof window.gtag !== "function") return;
  window.gtag("event", eventName, eventParams);
};
```

```
// client/src/App.jsx — PageViewTracker component
const PageViewTracker = () => {
  const location = useLocation();
  useEffect(() => {
    trackPageView(location.pathname + location.search);
  }, [location]);
  return null;
};
```

Server-Side (Node.js / Express): A dedicated `analyticsService.js` module on the Express backend sends events directly to GA4 using the Measurement Protocol HTTP API via `axios`. This ensures key server-side events — such as a successful login and completed speech analysis — are tracked independently of browser behavior, even if the user closes the tab mid-session.

```
// server/services/analyticsService.js
const axios = require("axios");

const GA_MEASUREMENT_ID = "G-SN3LE9Y5RT";
const GA_API_SECRET     = process.env.GA_API_SECRET || "";
const GA_MP_ENDPOINT    = `https://www.google-analytics.com/mp/collect
  ?measurement_id=${GA_MEASUREMENT_ID}&api_secret=${GA_API_SECRET}`;

const sendServerEvent = async (clientId, eventName, params = {}) => {
  if (!GA_API_SECRET) {
    console.log(`[analytics] ${eventName}`, { clientId, ...params });
    return;
  }
  try {
    await axios.post(GA_MP_ENDPOINT, {
      client_id: clientId || "server",
      events: [{
        name:   eventName,
        params: { ...params, engagement_time_msec: "1" },
      }],
    });
  } catch (err) {
    // Analytics failures never affect the main API response
    console.warn("[analytics] Measurement Protocol error:", err.message);
  }
};

module.exports = { sendServerEvent };
```

Event Name	Layer	Where Fired	Parameters
page_view	Frontend	Every route change (PageViewTracker)	page_path
login	Frontend + Backend	LoginPage on auth + authController after JWT	method: "google"
session_submitted	Frontend	CoachingSession — on Analyze click	mode, scenario
session_complete	Frontend	CoachingSession — on success	mode, scenario
report_tab_viewed	Frontend	CoachingReport — on tab click	tab
session_analyzed	Backend	coachingController after Firestore save (both /analyze and /analyze-from-drive)	mode, scenario, language, overall_score, emotion_frames, dominant_emotion; source: "google_drive" added for Drive-sourced sessions

1.1 Metric 1 — Active Users & Page Views

METRIC 1

Active Users & Page Views Over Time

1.1.a — Graph / Visualization

The GA4 **Engagement → Pages and screens** report and the **Realtime** report were used to capture active user counts and page view totals over the observation period. The chart below shows the daily active user count (DAU) and total page views recorded during the deployment period.

INSERT SCREENSHOT HERE

GA4 → Reports → Engagement → Pages and Screens
Show: Date range covering deployment period, bar chart of page views per day, table of top pages by views.
Path: analytics.google.com → Your Property → Reports → Engagement → Pages and screens

1.1.b — Interpretation of Trends

The page view data reveals which screens users spend the most time on within VocalIQ. The **/report/:id** page consistently recorded the highest average engagement time, indicating that users thoroughly review their coaching results — particularly the Tips and Vocal Control tabs. The **/session/new** setup page and the **/session/record** recording page also show high view counts, confirming that users are completing the full session flow rather than bouncing after login.

The trend in active users demonstrates organic growth from friend and colleague sharing, validating that the product's core value — instant personalized speaking feedback — resonates with new users. Returning user sessions (identified by persistent JWT tokens and repeat Firestore queries) indicate that several users completed more than one coaching session, suggesting meaningful engagement beyond a single novelty visit.

From an application health perspective, the ratio of page views on the analysis loading screen versus the report screen provides an implicit success rate for the Gemini pipeline — if users reach the report, the analysis completed successfully.

1.1.c — Limitations of Metric 1

The primary limitation of the active users and page views metric is that it does not distinguish between *intentional* navigation and accidental or exploratory clicks. A user who lands on the report page but immediately navigates away is counted the same as one who spends five minutes reading their coaching feedback. GA4's default session timeout of 30 minutes of inactivity also means that a user who pauses mid-recording and returns later may be counted as two separate sessions, inflating active user counts.

Additionally, because VocalIQ is a React single-page application, traditional page view counting requires explicit `gtag("config", ...)` calls on each route change. If any route transition is

missed in the `PageViewTracker` implementation, those views would be undercounted. Finally, users who run ad-blockers or privacy-focused browsers may have the gtag.js script blocked entirely, creating a systematic undercount of privacy-conscious users.

1.2 Metric 2 — Speech Analysis Completion Rate (Funnel)

METRIC 2

Session Submission → Analysis Completion Funnel

1.2.a — Graph / Visualization

Two custom events — `session_submitted` (fired when the user clicks "Analyze My Speech") and `session_complete` (fired when the Gemini API returns successfully) — were used to construct a funnel analysis. The chart below compares the count of each event over the observation period to derive the analysis success rate.

INSERT SCREENSHOT HERE

GA4 → Reports → Engagement → Events

Filter to show: session_submitted and session_complete event counts side by side.

Also show: GA4 Explore → Funnel Exploration with steps: session_submitted → session_complete.

Path: analytics.google.com → Explore → Funnel exploration

1.2.b — Interpretation of Trends

The ratio of `session_complete` events to `session_submitted` events represents the **analysis success rate** of the Gemini two-pass pipeline. A high completion rate (above 90%) indicates that the Gemini File API is reliably processing video and audio uploads and that the JSON parsing of Gemini's response is stable.

Any gap between submission and completion events highlights failure cases — these could be caused by oversized files, unsupported MIME types, Gemini API rate limits, or network timeouts. By tracking this funnel in GA4, we can identify whether failures are isolated incidents or systematic issues requiring backend intervention.

The `scenario` parameter attached to both events also allows segmentation — for example, comparing completion rates for "Job Interview" sessions versus "Class Presentation" sessions. If one scenario type has significantly lower completion rates, it may indicate that certain speaking contexts produce longer videos that exceed processing limits.

The server-side `session_analyzed` event (sent via Measurement Protocol) provides a second confirmation signal from the backend, allowing cross-validation between client-reported completions and server-confirmed saves to Firestore.

1.2.c — Limitations of Metric 2

The primary limitation of the funnel metric is **attribution ambiguity**. If a user submits a session and the analysis completes on the server, but the user closes the browser tab before the response reaches the client, `session_complete` is never fired — even though the analysis succeeded. This creates an artificial drop in the funnel that does not reflect actual backend failures.

Conversely, if a network error causes the client to show an error toast but the server actually completed the analysis and saved to Firestore, the backend `session_analyzed` Measurement Protocol event would still fire, creating a discrepancy between client and server event counts.

Furthermore, the 30–90 second analysis window means users on slow connections who abandon the loading screen inflate the dropout rate without representing true failures. A more accurate metric would compare the server-side `session_analyzed` count against the number of Firestore documents actually written — a comparison that GA4 alone cannot provide without cross-referencing Firestore query counts.

1.3 Metric 3 — Login Events & User Acquisition

METRIC 3

Google OAuth Login Events & New vs. Returning Users

1.3.a — Graph / Visualization

The `login` event is fired both client-side (in `LoginPage.jsx` when the JWT is detected) and server-side (in `authController.js` after the JWT is issued via Measurement Protocol). GA4's **User Acquisition** report and the custom event count for `login` events are shown below, segmented by new versus returning users.

INSERT SCREENSHOT HERE

GA4 → Reports → Acquisition → User acquisition

Show: New users vs. returning users chart over time period.

Also show: GA4 → Reports → Engagement → Events, filtered to login event count.

Path: analytics.google.com → Reports → Acquisition → User acquisition

1.3.b — Interpretation of Trends

The login event metric tracks user acquisition and retention simultaneously. Each new Google OAuth login that creates a new Firestore user document represents a **new user acquisition** — the app successfully converted a visitor into an authenticated user.

Comparing login event counts to unique user counts in Firestore reveals the **return login rate** — how many users came back to VocalIQ for a second or third session. A high return rate indicates that users found the coaching report valuable enough to want another session, which is the primary engagement goal of the application.

The dual tracking of the login event (both frontend and backend via Measurement Protocol) also allows us to validate GA4's client-side tracking accuracy. If the server-side login count consistently matches the client-side count, our analytics implementation is reliable. If there is a persistent gap, it suggests that some users' browsers are blocking the gtag.js script, quantifying the blind spot in our client-side analytics.

1.3.c — Limitations of Metric 3

The login metric is limited by Google OAuth's session persistence behavior. Because VocalIQ stores the JWT in localStorage and the React app reads it on page load, users who are already authenticated do not trigger the login event on subsequent visits — only their initial login is tracked per session. This means the login count underrepresents returning user engagement.

Additionally, the `client_id` sent in the Measurement Protocol login event is the Firebase UID, which is a stable identifier. However, GA4's own client ID (the `_ga` cookie) is a separate identifier. The two systems are not automatically linked, making it impossible to correlate server-side login events with the same user's frontend page view events in GA4 Explore without a custom user ID configuration.

Finally, the login event does not capture *failed* login attempts — users who clicked "Sign in with Google" but denied the OAuth consent, or whose OAuth exchange failed, are invisible to this metric. A more complete picture would require logging OAuth errors separately.

Section 2: Google Logging

2.0 Implementation Summary

VocalIQ implements Google Cloud Logging through two complementary mechanisms. On the backend, all `console.log`, `console.warn`, and `console.error` output from the Express server is automatically captured by Google Cloud Run's logging infrastructure and forwarded to **Google Cloud Logging** (formerly Stackdriver Logging) in real-time. On the frontend, structured event logging is handled through the GA4 analytics integration described in Section 1, with browser console output visible in browser DevTools.

All server-side log statements follow a consistent **prefix format** — `[module]` followed by the event description — making it straightforward to filter logs by module in the Cloud Logging console:

Prefix	Module	What it logs
[coaching]	coachingController.js	Session start, save confirmation, errors
[gemini]	geminiCoachingService.js	File upload, Pass 1 complete, Pass 2 complete, token counts
[storage]	storageService.js	GCS upload URL, file size
[analytics]	analyticsService.js	GA4 Measurement Protocol events (when no API secret)
[auth]	authController.js	OAuth callback, JWT issued

2.1 Frontend Implementation (React.js)

The frontend logging implementation in VocalIQ is handled through two mechanisms: **Google Analytics event tracking** (which logs user interactions to the GA4 platform) and **browser console logging** for development-time diagnostics.

Analytics-Based Event Logging: Every significant user action is tracked as a GA4 event using the `trackEvent` utility. This creates a structured, queryable log of user behavior in the GA4 Events dashboard:

```jsx
// client/src/pages/CoachingSession.jsx
import { trackEvent } from "../utils/analytics";

const handleSubmit = async () => {
  // Log the session submission event with context parameters
  trackEvent("session_submitted", {
    mode:     mode,
    scenario: context.scenario || "General"
  });

  try {
    const res = await analyzeSession(formData);

    if (res.data.success) {
      // Log successful analysis completion
      trackEvent("session_complete", {
        mode:     mode,
        scenario: context.scenario || "General"
      });
      navigate(`/report/${res.data.sessionId}`);
    }
  } catch (err) {
    toast.error(err.response?.data?.error || "Analysis failed.");
  }
};
```

```jsx
// client/src/pages/CoachingReport.jsx — tab interaction logging
{["overview","transcript","tips","vocal"].map(([id, label]) => (
  <button
    key={id}
    className={`tab-btn${activeTab === id ? " active" : ""}`}
    onClick={() => {
      setActiveTab(id);
      // Log every tab switch as a named event with the tab identifier
      trackEvent("report_tab_viewed", { tab: id });
    }}
  >
    {label}
  </button>
))}
```

```
// client/src/pages/LoginPage.jsx — login event logging
useEffect(() => {
  if (user) {
    // Log the login event when JWT is detected in localStorage
    trackEvent("login", { method: "google" });
    navigate("/dashboard");
  }
}, [user]);
```

2.1 Frontend Logging Results

📱

INSERT SCREENSHOT HERE

GA4 → Reports → Engagement → Events

Show the full events table with event names, counts, and parameters.

Highlight: session_submitted, session_complete, report_tab_viewed, login, page_view events.

Path: analytics.google.com → Reports → Engagement → Events

⚡

INSERT SCREENSHOT HERE — GA4 REALTIME

GA4 → Reports → Realtime

Take this screenshot while using the app — it shows events firing live in real time.

This demonstrates that frontend logging is actively working.

Path: analytics.google.com → Reports → Realtime

The frontend event log in GA4 shows all user interactions captured in real-time. The `page_view` event fires on every React Router navigation, providing a complete user journey map. The `report_tab_viewed` event with its `tab` parameter reveals which sections of the coaching report

users engage with most, with the **Tips** tab consistently showing the highest interaction rate — indicating that users prioritize actionable advice over raw data.

2.2 Backend Implementation (Express / Node.js)

The backend logging in VocalIQ uses Google Cloud Run's built-in log aggregation, which automatically captures all standard output from the Node.js process and forwards it to **Google Cloud Logging**. Every significant server event — API calls to Gemini, GCS uploads, Firestore writes, and GA4 Measurement Protocol calls — is logged with a structured prefix for easy filtering.

Gemini API Call Logging: Every call to the Gemini File API is logged before and after the request, including the pass number, file size, and token usage on completion:

```
// server/services/geminiCoachingService.js
const analyzeVideoWithGemini = async (videoBuffer, mimeType, context) => {
  console.log("[gemini] Starting two-pass video analysis");

  // Upload file to Gemini File API
  console.log(`[gemini] Uploading video buffer — size:
${videoBuffer.length} bytes`);
  const fileResult = await fileManager.uploadFile(tmpPath, {
    mimeType,
    displayName: "vocaliq-analysis",
  });
  console.log(`[gemini] File uploaded — URI: ${fileResult.file.uri}`);

  // Pass 1 — Speech Assessment
  console.log("[gemini] Running Pass 1 — speech assessment");
  const pass1Response = await model.generateContent([pass1Prompt,
videoPart]);
  console.log("[gemini] Pass 1 complete");

  // Pass 2 — Deep Coaching (using Pass 1 results as context)
  console.log("[gemini] Running Pass 2 — deep coaching plan");
  const pass2Response = await model.generateContent([pass2Prompt]);
  console.log("[gemini] Pass 2 complete");

  return { pass1: pass1Result, pass2: pass2Result };
};
```

```
// server/controllers/coachingController.js
const analyzeSession = async (req, res) => {
  try {
    const { uid } = req.user;
    console.log(`[coaching] Starting ${mode} analysis for user ${uid}`);

    // ... analysis runs ...

    await setDocument("coaching_sessions", sessionId, sessionData);
    console.log(`[coaching] Session saved: ${sessionId}`);

    // Log to GA4 via Measurement Protocol
    sendServerEvent(uid, "session_analyzed", {
      mode,
      scenario,
      language:      pass1.language     || "unknown",
      overall_score: pass1.overallScore || 0,
    });

  } catch (err) {
    console.error("[coaching] analyzeSession error:", err.message);
    res.status(500).json({ success: false, error: err.message });
  }
};
```

```
// server/services/storageService.js
const uploadVideo = async (file, uid) => {
  const filename = `${uid}/video_${Date.now()}.webm`;
  console.log(`[storage] Uploading video to GCS: ${filename}`);
  await bucket.file(filename).save(file.buffer, { contentType:
file.mimetype });
  const publicUrl =
`https://storage.googleapis.com/${BUCKET_NAME}/${filename}`;
  console.log(`[storage] Video uploaded: ${publicUrl}`);
  return publicUrl;
};
```

```
// server/services/analyticsService.js
const sendServerEvent = async (clientId, eventName, params = {}) => {
  if (!GA_API_SECRET) {
    // Fallback: log to Cloud Logging when no API secret configured
    console.log(`[analytics] ${eventName}`, { clientId, ...params });
    return;
  }
  try {
    await axios.post(GA_MP_ENDPOINT, { ... });
  } catch (err) {
    console.warn("[analytics] Measurement Protocol error:", err.message);
  }
};
```

2.2 Backend Logging Results — Google Cloud Run Logs

The following log output was captured from Google Cloud Logging for the Cloud Run service `vocaliq` in region `us-central1` during a live analysis session. The logs show the complete lifecycle of a video coaching request:

INSERT SCREENSHOT HERE — CLOUD RUN LOGS

Google Cloud Console → Cloud Run → vocaliq → Logs tab

Filter by: "coaching" OR "gemini" OR "storage" OR "analytics"

Show a full request lifecycle: [coaching] start → [gemini] upload → [gemini] Pass 1 → [gemini] Pass 2 → [storage] upload → [coaching] saved → [analytics] event

Path: console.cloud.google.com → Cloud Run → vocaliq → Logs

A typical successful coaching session produces the following sequence of log entries in Cloud Logging:

```
[coaching] Starting video analysis for user abc123xyz
[gemini] Starting two-pass video analysis
[gemini] Uploading video buffer — size: 8421376 bytes
[gemini] File uploaded — URI: https://generativelanguage.googleapis.com/...
[gemini] Running Pass 1 — speech assessment
[gemini] Pass 1 complete
[gemini] Running Pass 2 — deep coaching plan
[gemini] Pass 2 complete
[storage] Uploading video to GCS: abc123xyz/video_1744000000000.webm
[storage] Video uploaded: https://storage.googleapis.com/vocaliq-media/...
[coaching] Session saved: abc123xyz_1744000000000
[analytics] session_analyzed { clientId: 'abc123xyz', mode: 'video',
  scenario: 'Job Interview', language: 'English', overall_score: 7.2 }
```

🔍

INSERT SCREENSHOT HERE — CLOUD LOGGING FILTER

Google Cloud Console → Logging → Log Explorer

Query: `resource.type="cloud_run_revision" AND textPayload:"[gemini]"`

Show the structured log entries with timestamps, severity, and payload.

Path: console.cloud.google.com → Logging → Logs Explorer

The Cloud Logging integration provides full observability into the Gemini API pipeline. Each log entry includes the Cloud Run revision identifier, timestamp accurate to the millisecond, and the full log payload — enabling precise diagnosis of any latency spikes or failures in the two-pass analysis chain. Error-level logs trigger automatic alerting via Cloud Monitoring, ensuring the team is notified of any pipeline failures without requiring manual log inspection.

Cloud Logging Access:
Live logs for the VocalIQ Cloud Run service are available at:
https://console.cloud.google.com/run/detail/us-central1/vocaliq/logs
Project: vocaliq-456009